

Name: R.Prajwal

Hall ticket No: 2303A51830

Batch: 26

SCHOOL F COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week9 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 17.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts	Week9 - Wednesday																		

Lab Objectives:

- Learn how to clean raw datasets using AI-assisted Python scripting.
- Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization.
- Automate repetitive data-cleaning tasks with AI-generated code.
- Understand how preprocessing impacts model performance.

Task 1 Employee Dataset Cleaning

Task:

Use AI assistance to clean an employee information dataset.

Instructions:

- Handle missing values in salary, designation, and experience
- Remove duplicate employee IDs
- Standardize text values (job titles in lowercase)
- Encode categorical features (designation, employment_type)

Expected Output:

A clean dataset suitable for HR analytics and prediction tasks.

```
index.html | styles.css | script.js | layout.html | layoutstyles.css | dashboard | data | data.js | data.css | Skill_tracker.py | classnode.py X
C:\Users\pradeep> python language > classnode.py > ...
1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder
3
4 # Sample employee dataset (replace with your actual data loading)
5 data = {
6     'employee_id': [1, 2, 3, 4, 5, 1, 6],
7     'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', 'Frank'],
8     'salary': [50000, None, 60000, 55000, None, 50000, 65000],
9     'designation': ['Manager', 'Developer', None, 'manager', 'Analyst', 'Manager', 'developer'],
10    'experience': [5, 3, None, 4, 2, 5, 6],
11    'employment_type': ['Full-time', 'Part-time', 'Full-time', 'Contract', 'Full-time', 'Full-time', 'Part-time']
12 }
13
14 df = pd.DataFrame(data)
15
16 # Step 1: Remove duplicate employee IDs (keep the first occurrence)
17 df = df.drop_duplicates(subset='employee_id', keep='first')
18
19 # Step 2: Handle missing values
20 # For salary and experience, fill with median
21 df['salary'] = df['salary'].fillna(df['salary'].median())
22 df['experience'] = df['experience'].fillna(df['experience'].median())
23
24 # For designation: fill with mode (most frequent)
25 df['designation'] = df['designation'].fillna(df['designation'].mode()[0])
26
27 # Step 3: Standardize text values (job titles in lowercase)
28 df['designation'] = df['designation'].str.lower()
29
30 # Step 4: Encode categorical features
31 le_designation = LabelEncoder()
32 df['designation_encoded'] = le_designation.fit_transform(df['designation'])
33
34 le_employment = LabelEncoder()
35 df['employment_type_encoded'] = le_employment.fit_transform(df['employment_type'])
36
37 # Display the clean dataset
38 print(df)
39
40 # Optional: Save to CSV
41 df.to_csv('clean_employee_dataset.csv', index=False)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
Active code page: 65001
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
Active code page: 65001
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
employee_id name salary designation experience employment_type designation_encoded employment_type_encoded
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
0 1 Alice 50000.0 manager 5.0 Full-time 2 1
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
1 2 Bob 57500.0 developer 3.0 Part-time 1 2
2 3 Charlie 60000.0 analyst 4.0 Full-time 0 1
2 3 Charlie 60000.0 analyst 4.0 Full-time 0 1
3 4 David 55000.0 manager 4.0 Contract 2 0
4 5 Eve 57500.0 analyst 2.0 Full-time 0 1
6 6 Frank 65000.0 developer 6.0 Part-time 1 2
C:\Users\prade>
```

Task 2 – Student Performance Data Preprocessing

Task:

Preprocess student performance data using AI assistance.

Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

Expected Output:

A transformed dataset with time-based academic features and normalized marks.

```
index.html style.css scripts logou.html logoustyle.css dashboard ctags ctags ctags Skill_tracker.py
C:\Users\prade>OneDrive>Pradeep>python language>classnode.py ...
1
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import MinMaxScaler
5
6 # Sample Data
7 np.random.seed(42)
8 dates = pd.date_range('2023-01-01', periods=100, freq='D')
9 df = pd.DataFrame({
10     'student_id': range(1, 101),
11     'exam_date': np.random.choice(dates, 100),
12     'total_marks': np.random.normal(75, 15, 100),
13 })
14 df.loc[5, 'total_marks'] = 150 # Outlier
15
16 # 1. Convert to datetime
17 df['exam_date'] = pd.to_datetime(df['exam_date'])
18
19 # 2. Extract time features
20 df['month'] = df['exam_date'].dt.month
21 df['day'] = df['exam_date'].dt.day
22 df['semester'] = df['exam_date'].dt.month.apply(lambda x: 1 if x <= 6 else 2)
23
24 # 3. Outlier Detection - IQR Method
25 Q1, Q3 = df['total_marks'].quantile([0.25, 0.75])
26 IQR = Q3 - Q1
27 lower_bound = Q1 - 1.5 * IQR
28 upper_bound = Q3 + 1.5 * IQR
29 df['outlier'] = (df['total_marks'] < lower_bound) | (df['total_marks'] > upper_bound)
30
31 # 4. Outlier Detection - Z-Score Method
32 z_scores = np.abs((df['total_marks'] - df['total_marks'].mean()) / df['total_marks'].std())
33 df['z_outlier'] = z_scores > 3
34
35 # 5. Handle Outliers (Remove)
36 df = df[~df['outlier']].reset_index(drop=True)
37
38 # 6. Normalize Marks (Min-Max)
39 scaler = MinMaxScaler()
40 df['total_marks_normalized'] = scaler.fit_transform(df['total_marks'])
41
42 # Display Results
43 print("Preprocessed Data:")
44 print(df[['student_id', 'exam_date', 'month', 'semester', 'total_marks', 'total_marks_normalized']].head(10))
45 print(f"Original shape: {df.shape}, Processed shape: {df.shape}")
46 print(f"Normalized marks range: [{df['total_marks_normalized'].min():.4f}, {df['total_marks_normalized'].max():.4f}]")
47
48 # Save
49 df.to_csv(r'c:\Users\prade\OneDrive\Pradeep\python language\processed_student_data.csv', index=False)
50 print("\nProcessed data saved to processed_student_data.csv")
```

```
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Preprocessed Data:
student_id exam_date month semester total_marks total_marks_normalized
0 1 2023-02-21 2 1 54.799829 0.181188
1 2 2023-04-03 4 1 61.291131 0.306313
2 3 2023-01-15 1 1 58.841715 0.239348
3 4 2023-03-13 3 1 77.816433 0.578383
4 5 2023-03-02 3 1 83.731842 0.698443
5 6 2023-03-24 3 1 88.414985 0.782169
6 8 2023-03-28 3 1 86.324967 0.744884
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377781 0.137885

Original shape: (100,), Processed shape: (97, 9)
6 8 2023-03-28 3 1 86.324967 0.744884
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377781 0.137885

Original shape: (100,), Processed shape: (97, 9)
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377781 0.137885

Original shape: (100,), Processed shape: (97, 9)
Original shape: (100,), Processed shape: (97, 9)

Normalized marks range: [0.0000, 1.0000]
✓ Processed data saved to processed_student_data.csv
```

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

```
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
1 # Importing libraries
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder, train_test_split
5 from sklearn.model_selection import train_test_split
6
7 # Load the dataset
8 df = pd.read_csv('data/health_data.csv')
9
10 # Check for missing values
11 df.isnull().sum()
12
13 # Fill missing values
14 df['height'].fillna(df['height'].mean(), inplace=True)
15 df['weight'].fillna(df['weight'].mean(), inplace=True)
16 df['bmi'].fillna(df['bmi'].mean(), inplace=True)
17 df['medical_condition'].fillna('None', inplace=True)
18 df['fitness_status'].fillna('None', inplace=True)
19 df['exercise_hours'].fillna(0, inplace=True)
20
21 # Add missing values
22 missing_data = df.isnull().sum()
23 df.isnull().sum()
24 df['missing_data'].sum()
25
26 # Handle missing values
27 df['height'].fillna(df['height'].mean(), inplace=True)
28 df['weight'].fillna(df['weight'].mean(), inplace=True)
29 df['bmi'].fillna(df['bmi'].mean(), inplace=True)
30 df['medical_condition'].fillna('None', inplace=True)
31 df['fitness_status'].fillna('None', inplace=True)
32 df['exercise_hours'].fillna(0, inplace=True)
33
34 # Encode categorical features
35 le_medical = LabelEncoder()
36 le_fitness = LabelEncoder()
37 df['medical_encoded'] = le_medical.fit_transform(df['medical_condition'])
38 df['fitness_encoded'] = le_fitness.fit_transform(df['fitness_status'])
39
40 # Scale numerical features
41 numerical_data = df[['age', 'height', 'weight', 'bmi', 'exercise_hours']]
42 scaler = StandardScaler()
43 df[numerical_data.columns] = scaler.fit_transform(df[numerical_data])
44
45 # Split dataset
46 X = df[numerical_data.columns]
47 y = df[['medical_encoded', 'fitness_encoded']]
48 y = pd.get_dummies(y, columns=['medical_encoded', 'fitness_encoded'])
49
50 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
51
52 # Print the results
53 print("Before preprocessing:")
54 print(df.isnull().sum())
55 print("After preprocessing:")
56 print(df.isnull().sum())
57 print("Training set: ({} rows, {} features), Testing set: ({} rows, {} features)".format(X_train.shape[0], X_train.shape[1], X_test.shape[0], X_test.shape[1]))
58
59 # Save the data
60 df.to_csv('data/processed_health_data.csv', index=False)
61 X_train.to_csv('data/X_train.csv', index=False)
62 X_test.to_csv('data/X_test.csv', index=False)
63 y_train.to_csv('data/y_train.csv', index=False)
64 y_test.to_csv('data/y_test.csv', index=False)
65
66 print("Data saved: processed_health_data.csv, health_train.csv, health_test.csv")
```

```
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Before preprocessing:
Missing values:
student_id      0
age              0
height          3
weight          3
bmi             4
medical_condition 21
fitness_status   25
exercise_hours   0
dtype: int64
```

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df['fitness_status'].fillna('Fair', inplace=True)
After preprocessing:
Missing values: 56
Encoded features: 2
Scaled features: 5

Training set: 80, Testing set: 20
```

	age	height	weight	bmi	exercise_hours	medical_encoded	fitness_encoded
55	1.465198	0.306641	0.023128	0.300740	-0.473295	1	2
88	0.414877	-1.596923	-0.885985	-0.042759	0.560665	3	1
26	-0.110284	0.412532	-0.387245	-0.502634	0.135869	3	4
42	1.465198	-0.817847	-0.364701	2.263329	-1.788820	1	0
69	1.465198	0.004662	1.032667	1.151545	-1.403043	2	2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv

After preprocessing:

Missing values: 56

Encoded features: 2

Scaled features: 5

Training set: 80, Testing set: 20

	age	height	weight	bmi	exercise_hours	medical_encoded	fitness_encoded
55	1.465198	0.306641	0.023128	0.300740	-0.473295	1	2
88	0.414877	-1.596923	-0.885985	-0.042759	0.560665	3	1
26	-0.110284	0.412532	-0.387245	-0.502634	0.135869	3	4
42	1.465198	-0.817847	-0.364701	2.263329	-1.788820	1	0
69	1.465198	0.004662	1.032667	1.151545	-1.403043	2	2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv

42 1.465198 -0.817847 -0.364701 2.263329 -1.788820 1 0

69 1.465198 0.004662 1.032667 1.151545 -1.403043 2 2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv

Task 4: Real-Time Application: Ride-Sharing Data Cleaning Scenario:

A ride-sharing company has trip data with errors.

Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

```
C:\Users\prade> CodeRunner > Run (Python 3.10.0)
1 # Create Ride-Sharing Data
2 np.random.seed(42)
3
4 data = {
5     'ride_id': range(1, 100),
6     'pickup_location': np.random.choice(['Downtown', 'AIRPORT', 'Downtown', 'airport', 'Home', 100]),
7     'drop_location': np.random.choice(['Downtown', 'AIRPORT', 'Downtown', 'airport', 'Home', 100]),
8     'fare_amount': np.random.normal(10, 10),
9     'driver_rating': np.random.uniform(1, 5, 100),
10 }
11
12 df = pd.DataFrame(data)
13
14 # Add duplicates and missing values
15 df.loc[random_row] = np.random.choice(100, 5, replace=False), 'fare_amount' = np.NaN
16 df = pd.concat([df, df.iloc[15]], ignore_index=True)
17
18 print("RIDE-SHARING DATA CLEANING PIPELINE")
19 print("="*40)
20 print(f"Initial dataset shape: {df.shape}")
21 print(f"Missing values: {df.isnull().sum()}")
22
23 # 1. Standardize Locations
24 df['pickup_location'] = df['pickup_location'].str.strip().str.lower().str.title()
25 df['drop_location'] = df['drop_location'].str.strip().str.lower().str.title()
26 df['pickup_location'].fillna('Unknown', inplace=True)
27 df['drop_location'].fillna('Unknown', inplace=True)
28 print(f"Standardized pickup and drop locations")
29
30 # 2. Fill Missing Fare with Median
31 median_fare = df['fare_amount'].median()
32 df['fare_amount'].fillna(median_fare, inplace=True)
33 print(f"Filled missing fare with median: {median_fare:.2f}")
34
35 # 3. Remove Duplicates
36 initial_row = len(df)
37 df = df.drop_duplicates(subset=['ride_id', 'pickup_location', 'drop_location'], keep='first')
38 df = df.reset_index(drop=True)
39 duplicates_removed = initial_row - len(df)
40 print(f"Removed duplicate entries: {duplicates_removed} rows")
41
42 # 4. Normalize Driver Ratings (0-1 scale)
43 scaler = MinMaxScaler()
44 df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])
45 print(f"Normalized driver ratings (0-1 scale)")
46
47 # 5. Generate Summary
48 print("\n" + "="*40)
49 print("DATA QUALITY IMPROVEMENT SUMMARY")
50 print("="*40)
51 print(f"Final dataset shape: {df.shape}")
52 print(f"Duplicate entries removed: {duplicates_removed}")
53 print(f"Missing fare filled: {(df['fare_amount'].isnull().sum() == 0)}")
54 print(f"Location standardization: Complete")
55 print(f"Driver ratings normalized: Complete")
56 print(f"Variance Statistics:")
57 print(f"Min: {df['fare_amount'].min():.2f}")
58 print(f"Max: {df['fare_amount'].max():.2f}")
59 print(f"Mean: {df['fare_amount'].mean():.2f}")
60 print(f"Median: {df['fare_amount'].median():.2f}")
```

```
index.html style.css script.js layout.html layoutstyle.css cta.html cta.js cta.css Skill_tracker.py classnode.py X
C:\Users\prade> OneDrive > Pradeep > python language > classnode.py > ...

64
65 # Save cleaned data
66 df.to_csv('c:/Users/prade/OneDrive/Pradeep/python language/cleaned_rides.csv', index=False)
67 print("\n/ Clean dataset saved to cleaned_rides.csv")
68 print("\nCleaned Data Sample:\n(df.head())")
69

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL-CHECKER

C:\Users\prade> C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
RIDE-SHARING DATA CLEANING PIPELINE
=====

Original shape: (185, 5)
Missing values:
ride_id      0
pickup_location 22
drop_location 17
fare_amount   5
driver_rating 0
dtype: int64

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
df[['fare_amount']].fillna(method='bfill', inplace=True)
/ Filled missing fares with median: $25.48
/ Removed duplicate entries: 5 rows
/ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.30
Median: $25.28
df[['fare_amount']].fillna(method='bfill', inplace=True)
/ Filled missing fares with median: $25.48
/ Removed duplicate entries: 5 rows
/ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.30
Median: $25.28
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.30
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.30
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.30
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Cleaned Data Sample:

Cleaned Data Sample:
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
0 1 Airport Midtown 24.545971 2.312611 0.323680
0 1 Airport Midtown 24.545973 1.620166 0.323680
1 2 NaN Midtown 27.169367 1.620166 0.144575
2 3 Downtown Midtown 30.124237 4.927364 1.000000
1 2 NaN Midtown 27.169367 1.620166 0.144575
2 3 Downtown Midtown 30.124237 4.927364 1.000000
3 4 NaN Station 30.434887 4.355734 0.852345
4 5 NaN Midtown 25.288955 4.441618 0.874359
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.